

Arquitectura Híbrida de Recuperación Aumentada por Generación (RAG) con Grafo de Conocimiento

Team Kepler

Karen Julieth Castaño Trujillo

Luis David Sánchez Salazar

Luis Jeronimo Lopez Guterrez

Samuel Camilo Merchan Macedo

15 de agosto de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Metodología y Flujo de Trabajo	2
3.1. Roles y Distribución del Trabajo	2
3.1.1. Módulo 1: Extracción e Ingesta de Documentos Multimodal .	3
3.1.2. Almacén y Gestión de Metadatos	3
3.1.3. Estrategia de Chunking	3
3.1.4. Módulo 2: Preprocesamiento de Texto y Chunking	4
3.1.5. Estrategia de Chunking	5
3.1.6. Módulo 3: Indexación Vectorial y Persistencia	5
3.1.7. Módulo 4: Grafo de Conocimiento y Fusión RRF	6

1. Introducción

En el marco del presente reto tecnológico, este informe detalla la conceptualización, diseño y despliegue de un pipeline de Recuperación Aumentada por Generación (RAG). El objetivo central fue superar las limitaciones de la búsqueda vectorial tradicional mediante la integración de un Grafo de Conocimiento semántico. Esta arquitectura híbrida permite no solo recuperar información basada en la similitud de contextos multilingües, sino también priorizar documentos mediante relaciones topológicas exactas entre entidades clave detectadas en las consultas de los usuarios.

2. Objetivos

2.1. Objetivo General

Desarrollar e implementar un sistema RAG híbrido, modular y escalable que fusione técnicas de recuperación vectorial (FAISS) con la precisión de un Grafo de Conocimiento estructurado, optimizando la extracción de información en un corpus de documentos multilingüe.

2.2. Objetivos Específicos

- Extraer y procesar texto e imágenes de un corpus documental aplicando técnicas de OCR y segmentación (chunking) controlada (máximo 250 palabras por fragmento).
- Generar embeddings multilingües de alta calidad e indexarlos utilizando la librería FAISS para búsquedas por similitud semántica.
- Construir un Grafo de Conocimiento extrayendo entidades nombradas (NER) mediante modelos de procesamiento de lenguaje natural (Hugging Face) e identificando sus relaciones de co-ocurrencia.
- Implementar el algoritmo de fusión de rangos recíprocos (RRF) para combinar los scores vectoriales y topológicos, retornando los resultados en un formato JSONL estricto.

3. Metodología y Flujo de Trabajo

Para garantizar la agilidad y modularidad del proyecto, el equipo adoptó un flujo de trabajo colaborativo centralizado en GitHub. El desarrollo se estructuró en módulos independientes ejecutados secuencialmente a través de un orquestador principal (`main.py`). Esto permitió el trabajo en paralelo y la validación aislada de cada componente antes de su integración final.

3.1. Roles y Distribución del Trabajo

A continuación, se detallan las responsabilidades asumidas por cada integrante del equipo, reflejando la arquitectura técnica del pipeline:

3.1.1. Módulo 1: Extracción e Ingesta de Documentos Multimodal

Archivo responsable: `extractor.py`

Encargado del procesamiento inicial de la ingesta documental en el corpus multimodal. Implementó la lectura automatizada de archivos de texto y la integración de Reconocimiento Óptico de Caracteres (OCR) mediante `pytesseract` para digitalizar imágenes e infografías. Su desarrollo aseguró la extracción limpia de caracteres y estructuró la carga inicial de datos crudos para los pipelines posteriores.

3.1.2. Almacén y Gestión de Metadatos

Para extraer y gestionar eficientemente los metadatos de los documentos y sus posteriores fragmentos (*chunks*), la estrategia se basó en el uso de estructuras de datos clave-valor (diccionarios), permitiendo el acceso rápido a la información a través de sus claves:

- **Fase de Extracción (extractor):**
 - La función `procesar_documento` extrae el texto en bruto junto con sus metadatos base (`fuelle`, `formato`, `fenómeno`, `idioma`).
 - La función `procesar_carpeta` consolida estos resultados en una lista estructurada con un diccionario por cada documento analizado.
- **Fase de Chunking (procesador_texto):**
 - A partir de la lista producida por el extractor, la función `chunkers_final` genera un **diccionario global de diccionarios**. En esta estructura, cada fragmento queda completamente accesible a través de su identificador único (`chunk_id`), encapsulando tanto su texto limpiado como sus metadatos asociados.

3.1.3. Estrategia de Chunking

La estrategia de fragmentación atravesó una evolución lógica basada en la búsqueda de equilibrio entre densidad semántica y límites técnicos:

1. Hipótesis Inicial y Justificación Semántica Inicialmente, basándonos en estimaciones de densidad informacional en textos científicos y técnicos, consideramos que una longitud de **5 a 6 oraciones** representaba el tamaño óptimo para un *chunk*. Esta aproximación buscaba empaquetar una idea completa continuo-semántica manteniendo, en promedio, un conteo inferior al umbral de **250 tokens**.

2. Desafíos Identificados Aunque la hipótesis de 6 oraciones mantenía la cohesión de las ideas, identificamos dos limitaciones críticas en su aplicación rígida:

- **Falta de solapamiento (*overlap*):** Pérdida del hilo conductor contextual entre los límites de un fragmento y el siguiente.
- **Anomalías de longitud:** Existencia de oraciones atípicas con una cantidad de tokens tan elevada que desbordaban el límite permitido.

3. Arquitectura de Solución Implementada Para solucionar estos problemas sin renunciar a nuestra hipótesis principal, implementamos un sistema de filtrado recursivo y partición dinámica:

- **Filtro de Seguridad 1 (Omisión de oraciones anómalas):** Se evalúa individualmente la cantidad de tokens por oración. Si una oración supera por sí sola los 250 tokens, se emite una advertencia, se omite y se divide el párrafo en bloques seguros antes y después de la anomalía.
- **Filtro de Seguridad 2 (Aislamiento por suma adyacente):** Analiza la suma de tokens de cada oración con sus vecinas (anterior y posterior). Si ambas sumas superan los 250 tokens pero la oración individual no lo hace, esta se aísla como un fragmento independiente, dividiendo el texto circundante en bloques seguros.
- **Ventana Deslizante con Reducción Dinámica (*Backoff*) y Solapamiento:** Sobre los bloques seguros resultantes, el sistema intenta construir *chunks* respetando el techo ideal de 6 oraciones. Si un fragmento supera el límite de 250 tokens, **reduce progresivamente la cantidad máxima de oraciones** ($6 \rightarrow 5 \rightarrow 4 \dots$) hasta hacer cumplir la restricción. Finalmente, cada avance de ventana preserva un solapamiento de **1 oración** con el fragmento anterior, garantizando la preservación del hilo conductor.

3.1.4. Módulo 2: Preprocesamiento de Texto y Chunking

Archivo responsable: `procesador_texto.py`

Responsable de la limpieza, normalización de texto y segmentación estratégica de la información. Diseñó el algoritmo de *chunking* condicional que garantiza un límite estricto de máximo 250 palabras por fragmento. Asimismo, enriqueció cada bloque mediante la asignación metódica de identificadores de control (`doc_id` y `chunk_id`) e inyección de metadatos estructurales requeridos por el sistema RAG. Para extraer y gestionar eficientemente los metadatos de los documentos y sus posteriores fragmentos (*chunks*), la estrategia se basó en el uso de estructuras de datos clave-valor (diccionarios), permitiendo el acceso rápido a la información a través de sus claves:

- **Fase de Extracción (extractor):**
 - La función `procesar_documento` extrae el texto en bruto junto con sus metadatos base (`fuentes`, `formato`, `fenómeno`, `idioma`).
 - La función `procesar_carpeta` consolida estos resultados en una lista estructurada con un diccionario por cada documento analizado.
- **Fase de Chunking (procesador_texto):**
 - A partir de la lista producida por el extractor, la función `chunkers_final` genera un **diccionario global de diccionarios**. En esta estructura, cada fragmento queda completamente accesible a través de su identificador único (`chunk_id`), encapsulando tanto su texto limpiado como sus metadatos asociados.

3.1.5. Estrategia de Chunking

La estrategia de fragmentación atravesó una evolución lógica basada en la búsqueda de equilibrio entre densidad semántica y límites técnicos:

1. Hipótesis Inicial y Justificación Semántica Inicialmente, basándonos en estimaciones de densidad informacional en textos científicos y técnicos, consideramos que una longitud de **5 a 6 oraciones** representaba el tamaño óptimo para un *chunk*. Esta aproximación buscaba empaquetar una idea completa continuo-semántica manteniendo, en promedio, un conteo inferior al umbral de **250 tokens**.

2. Desafíos Identificados Aunque la hipótesis de 6 oraciones mantenía la cohesión de las ideas, identificamos dos limitaciones críticas en su aplicación rígida:

- **Falta de solapamiento (*overlap*):** Pérdida del hilo conductor contextual entre los límites de un fragmento y el siguiente.
- **Anomalías de longitud:** Existencia de oraciones atípicas con una cantidad de tokens tan elevada que desbordaban el límite permitido.

3. Arquitectura de Solución Implementada Para solucionar estos problemas sin renunciar a nuestra hipótesis principal, implementamos un sistema de filtrado recursivo y partición dinámica:

- **Filtro de Seguridad 1 (Omisión de oraciones anómalas):** Se evalúa individualmente la cantidad de tokens por oración. Si una oración supera por sí sola los 250 tokens, se emite una advertencia, se omite y se divide el párrafo en bloques seguros antes y después de la anomalía.
- **Filtro de Seguridad 2 (Aislamiento por suma adyacente):** Analiza la suma de tokens de cada oración con sus vecinas (anterior y posterior). Si ambas sumas superan los 250 tokens pero la oración individual no lo hace, esta se aísla como un fragmento independiente, dividiendo el texto circundante en bloques seguros.
- **Ventana Deslizante con Reducción Dinámica (*Backoff*) y Solapamiento:** Sobre los bloques seguros resultantes, el sistema intenta construir *chunks* respetando el techo ideal de 6 oraciones. Si un fragmento supera el límite de 250 tokens, **reduce progresivamente la cantidad máxima de oraciones** ($6 \rightarrow 5 \rightarrow 4 \dots$) hasta hacer cumplir la restricción. Finalmente, cada avance de ventana preserva un solapamiento de **1 oración** con el fragmento anterior, garantizando la preservación del hilo conductor.

3.1.6. Módulo 3: Indexación Vectorial y Persistencia

Archivo responsable: `indexador.py`

Encargado del diseño e implementación del motor de representación vectorial denso. Utilizó el modelo `sentence-transformers/distiluse-base-multilingual-cased-v1` para codificar los fragmentos (*chunks*) generados en vectores densos de 512 dimensiones, garantizando una alineación semántica multilingüe eficiente. Construyó e

indexó los vectores en una base de datos vectorial FAISS mediante un índice de producto interno (`faiss.IndexFlatIP`) con vectores normalizados $L2$, permitiendo búsquedas rápidas por similitud de coseno. Asimismo, garantizó la persistencia del sistema exportando la base vectorial en el archivo `index.faiss` y sincronizando los metadatos completos y el contenido textual de cada *chunk* en la base de metadatos `metadata.jsonl`.

3.1.7. Módulo 4: Grafo de Conocimiento y Fusión RRF

Archivos responsables: `constructor_grafo.py`, `generador.py`

Encargado del componente híbrido del sistema. Implementó un pipeline NER multilingüe para construir el archivo `grafo.graphml` mapeando entidades (Organizaciones, Locaciones, Personas). Asimismo, desarrolló el algoritmo de recuperación final, cruzando las predicciones de FAISS con los vecinos del grafo mediante Fusión RRF y aplicando *Max Pooling* a nivel de documento.

Para dotar al sistema de precisión semántica y entendimiento de entidades clave, se construyó un Grafo de Conocimiento dirigido utilizando la librería `networkx` de Python. El proceso de construcción se dividió en las siguientes fases técnicas:

- **Extracción de Entidades Multilingüe:** Se integró la librería `transformers` de Hugging Face empleando el modelo `Babelscape/wikineural-multilingual-ner`. Este modelo procesa cada fragmento de texto (*chunk*) y detecta entidades nombradas con un umbral de confianza estricto ($> 0,70$), agrupando tokens fragmentados mediante la estrategia *simple aggregation*.
- **Definición de Topología mediante `networkx`:** Se instanció un grafo dirigido (`nx.DiGraph()`). Por cada *chunk* analizado, las entidades únicas detectadas se insertaron utilizando la función `grafo.add_node()`, almacenando su tipo (e.g., ORG, LOC, PER) como atributo.
- **Generación de Relaciones:** Las aristas se establecieron asumiendo proximidad semántica dentro del límite de 250 palabras del fragmento. Mediante la función `grafo.add_edge()`, se vincularon las entidades co-ocurrentes bajo la relación `CO_OCURRE_EN_CHUNK`, inyectando directamente como metadatos obligatorios el `doc_id` y el `chunk_id` correspondientes.
- **Motor de Generación y Recuperación Híbrida (`generador.py`):** El script central de consultas orquesta la recuperación de información fusionando bases de datos vectoriales y topológicas. Es importante enfatizar que esta arquitectura se diseñó bajo un paradigma strictly **Encoder-only**. Para el procesamiento de las consultas en lenguaje natural, no se hizo uso de modelos generativos o decodificadores (LLMs). En su lugar, se utilizó el modelo `sentence-transformers/distiluse-base-multilingual-cased-v1`, el cual actúa puramente como un codificador (encoder) para mapear tanto el corpus como las consultas a un espacio vectorial denso de 512 dimensiones, garantizando una alineación semántica multilingüe de alta eficiencia y bajo costo computacional. El pipeline de recuperación integrado opera mediante las siguientes etapas:

1. **Recuperación Vectorial (FAISS):** El codificador transforma la consulta entrante en un vector numérico normalizado. A través de la librería `faiss`, se realiza una búsqueda de los $k = 60$ vecinos más cercanos midiendo la distancia $L2$.
2. **Recuperación Topológica (Grafo):** Paralelamente, la consulta es procesada por el pipeline NER. Las entidades identificadas se buscan en el `grafo.graphml`. Si existe coincidencia, el algoritmo explora los vecinos de primer orden utilizando las funciones `grafo.successors()` y `grafo.predecessors()`. Posteriormente, se itera sobre los atributos de las aristas conectadas para recuperar y contabilizar las frecuencias de los `chunk_id` implicados.
3. **Fusión de Resultados mediante RRF:** Para unificar los candidatos de FAISS y del Grafo sin depender de distribuciones de distancia heterogéneas, se aplicó la técnica de Reciprocal Rank Fusion (RRF). El puntaje combinado para cada fragmento c se calculó mediante la fórmula:

$$s_{RRF}(c) = \sum_{j=1}^m \frac{1}{k_0 + r_j(c)}$$

Donde k_0 se definió empíricamente con un valor de 60, y $r_j(c)$ representa la posición (rango) del fragmento en cada lista de recuperación (FAISS y Grafo).

4. **Agregación y Max Pooling:** Los fragmentos ordenados por su puntaje RRF son sometidos a un proceso de *Max Pooling* a nivel de documento. El sistema agrupa los *chunks* por `doc_id` y hereda la máxima puntuación encontrada. Finalmente, se trunca el texto de los fragmentos al límite normativo de 250 palabras y se exporta el top 3 de documentos en formato JSONL.